ConstructPro

Software Design Specification (SDS)

By

Iddamalgoda I.H.R.P.
SE/2021/038
Dhanushan K.I.
SE/2021/027
Ranasundara R.I.K.
SE/2021/001
Abeywadena M.B.
SE/2021/037

A report submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science Honors in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit
Faculty of Science
University of Kelaniya
Sri Lanka
2026

Version History

Version	Date	Author(s)	Description
1.0	April 2026	ConstructPro Team	Initial SDS draft
1.1	October 2026	ConstructPro Team	Design updated to reflect scope changes; class model and deployment details revised

Table of Contents

List of Tables iv

List of Figures v

Chapter 1 Architectural Design 1

1.1 High-level architecture 1

1.2 Cloud-Native Architecture 2

1.3 Justification for architecture choice 3

1.4 Scalability & maintainability considerations 3

Chapter 2 Database Design 5

2.1 ER Diagram 5

2.2 Normalization explanation 6

Chapter 3 Class Diagram 13

3.1 Attributes 14

3.2 Methods 15

3.3 Relationships 16

3.4 Visibility 16

3.5 Multiplicity 16

Chapter 4 Sequence Diagrams 18

4.1 Authentication & Login Sequence Diagram 18

4.2 Lead Capture & Quotation Generation Sequence Diagram 18

4.3 Quotation Approval & Project Creation Sequence Diagram 19

4.4 Payment Recording & Invoice Management Sequence Diagram 19

4.5 AI Rist Prediction Sequence Diagram 20

Chapter 5 UI/UX Design 21

5.1 Wireframes or screen mock-ups for key user interfaces 21

5.2 User flow / navigation diagrams 26

5.3 Accessibility considerations 32

Chapter 6 Security Design 34

6.1 Authentication mechanism 34

6.2 Authorization levels 35

6.3 Data protection approach 36

6.4 Input validation and sanitization strategy 36

6.5 Encryption standards for data at rest and in transit 37
Chapter 7 Deployment Design 38
7.1 Hosting Plan 38
7.2 Hardware Requirements 39
7.3 Network Diagram 40
7.4 Environment Specification 41
Chapter 8 References 43

List of Tables

Table 1: Role Table 7
Table 2: User Table 7
Table 3: Lead Table 8
Table 4: Quotation_Item Table 8
Table 5: Task Table 9
Table 6: Expense Table 9
Table 7: User Table 10
Table 8: Role Table 10
Table 9: Lead Table 10
Table 10: Customer Table 10
Table 11: Project Table 11
Table 12: Quotation Table 11
Table 13: Milestone Table 11
Table 14: Task Table 11
Table 15: Elimination of Data Redundancy 12

List of Figures

Figure 1: Architectural Design 1
Figure 2: ER Diagram 5
Figure 3: Class Diagram 13
Figure 4: Authentication & Login Sequence Diagram 18
Figure 5: Lead Capture & Quotation Generation Sequence Diagram 18
Figure 6: Quotation Approval & Project Creation Sequence Diagram 19
Figure 7: Payment Recording & Invoice Management Sequence Diagram 19
Figure 8: AI Rist Prediction Sequence Diagram 20
Figure 9: Dashboard UI 21
Figure 10: Lead Management UI 22
Figure 11: Quotations UI 22
Figure 12: Projects UI 23
Figure 13: Invoices & Payments UI 23
Figure 14: Document Management UI 24
Figure 15: Analytics & AI Risk Prediction UI 24
Figure 16: User Management 25
Figure 17: Role-based user flow diagram 26
Figure 18: Lead-quotation-project flow diagram 27
Figure 19: Project management flow 28
Figure 20: Finance and payment management flow 29
Figure 21: Analytics and AI risk prediction flow 30
Figure 22: Document management flow 31
Figure 23: Authentication mechanism 34
Figure 24: Authorization levels 35
Figure 25: Hosting Plan 38
Figure 26: Network Diagram 40

Chapter 1 Architectural Design

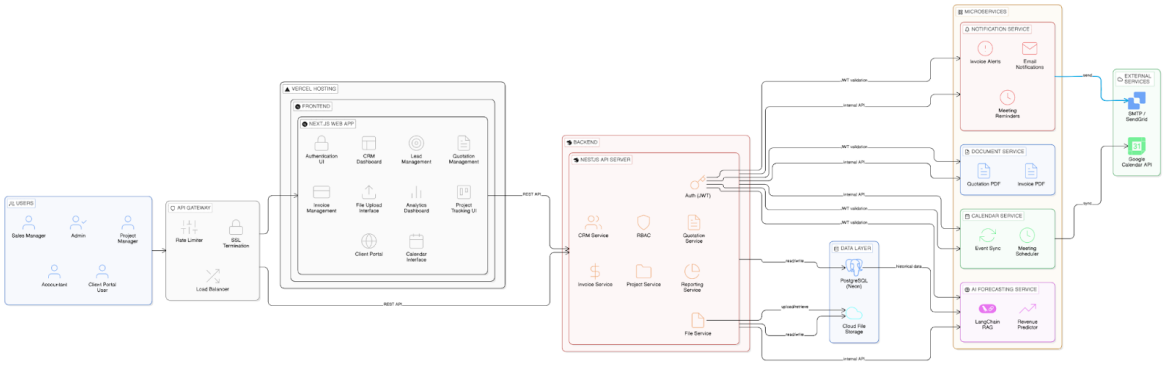


Figure 1: Architectural Design

1.1 High-level architecture

The ConstructPro ERP system has been designed using a hybrid layered architecture with supporting microservices. The architecture separates the system into multiple logical layers including the user layer, API gateway layer, presentation layer, backend service layer, microservices layer, data layer, and external service integrations.

Figure 1 illustrates the high-level system architecture of ConstructPro ERP.

At the user layer, different user roles such as administrators, sales managers, project managers, accountants, and client portal users access the system through web browsers.

Requests from users first pass through the API Gateway, which provides essential infrastructure services including load balancing, rate limiting, and SSL termination. This layer acts as the primary entry point to the system and ensures secure and controlled access to backend services.

The frontend layer is implemented using a Next.js web application. This layer provides the user interfaces for authentication, CRM management, lead management, quotation and invoice handling, project tracking, analytics dashboards, file uploads, client portal functionality, and calendar interaction.

The backend layer consists of a centralized **NestJS API server** responsible for implementing the core business logic of the ERP system. Core backend modules include authentication

using JWT, role-based access control (RBAC), CRM services, quotation management, invoice processing, project tracking, reporting services, and file management. [4]

The architecture includes domain-specific backend services for document management, calendar integration, and AI-based forecasting. These services communicate through REST APIs and support the corresponding business workflows of the ERP system. [1]

The data layer consists of a **PostgreSQL database hosted on Neon** for storing application data such as users, leads, projects, quotations, invoices, and payments. Cloud file storage is used to store uploaded documents and generated PDF files. [5]

Finally, the system integrates with the Google Calendar API for meeting scheduling and calendar synchronization. [6]

1.2 Cloud-Native Architecture

ConstructPro ERP follows a cloud-native architecture where system components are deployed on cloud infrastructure and communicate through REST-based APIs.

The API Gateway serves as the main entry point for client requests and performs important infrastructure functions such as SSL termination, request routing, load balancing, and rate limiting. This ensures secure and scalable handling of incoming traffic.

The Next.js frontend application is deployed on **Vercel**, which provides optimized hosting for Next.js applications. The **NestJS** backend services operate as scalable application servers capable of handling REST API requests from the frontend layer. [2] [4]

Backend services are deployed on Vercel using Node.js serverless functions. The services support specific business operations, including document management, calendar integration, and AI-assisted predictive analysis. Communication between the API Gateway and backend services is handled through REST APIs.

The system uses **PostgreSQL hosted on Neon** as the primary cloud database for storing structured business data. Cloud file storage is used for managing uploaded documents, quotation files, and invoice PDFs. [5]

The Google Calendar API is accessed through a secure integration to support configured calendar scheduling functionality. [6]

The use of containerized services and API-based communication allows the system to be deployed across scalable cloud environments while maintaining flexibility for future expansion.

1.3 Justification for architecture choice

The hybrid layered architecture combined with supporting microservices was selected for ConstructPro ERP to balance system simplicity, modularity, and scalability.

A layered architecture was chosen because it provides a clear separation between presentation, application logic, and data management. This separation improves system organization and makes the platform easier to develop, maintain, and extend.

The centralized **NestJS** backend simplifies the implementation of core ERP workflows such as CRM management, quotation processing, project tracking, and reporting. These modules share common data structures and business logic, making a centralized backend architecture more efficient than distributing them across multiple independent services.

However, document generation, calendar synchronization, and AI forecasting have different processing characteristics. These functions are separated from the core ERP workflows to support modular development and maintenance.

The API Gateway provides centralized request management, security enforcement, and traffic control, ensuring that backend services remain protected from excessive or malicious requests.

This architectural design allows the system to maintain simplicity for core business operations while supporting modular expansion and integration with external services.

1.4 Scalability & maintainability considerations

The architecture of ConstructPro ERP has been designed to support both scalability and maintainability.

Scalability is achieved through the use of an API Gateway and load balancing infrastructure, which enables the system to distribute incoming requests across multiple backend instances when system demand increases. This allows the application to handle increased user traffic without affecting system performance.

The separation of specialized functionality also improves scalability. Document generation, calendar synchronization, and AI forecasting can be scaled according to workload requirements.

Maintainability is enhanced through the modular structure of the backend application. Each functional area such as CRM management, invoice processing, project tracking, and reporting is implemented as a separate service module within the **NestJS** backend. This modular design allows developers to update or modify individual modules without impacting the entire system.

The use of modern frameworks such as Next.js for the frontend and **Node.js with NestJS** for backend services further supports maintainability by enforcing a clear separation between presentation logic and server-side business logic. [3] [4]

Additionally, the use of **PostgreSQL on Neon** for structured data storage and cloud-based file storage for document management enables efficient handling of growing business data while maintaining system performance. [5]

Chapter 2 Database Design

2.1 ER Diagram

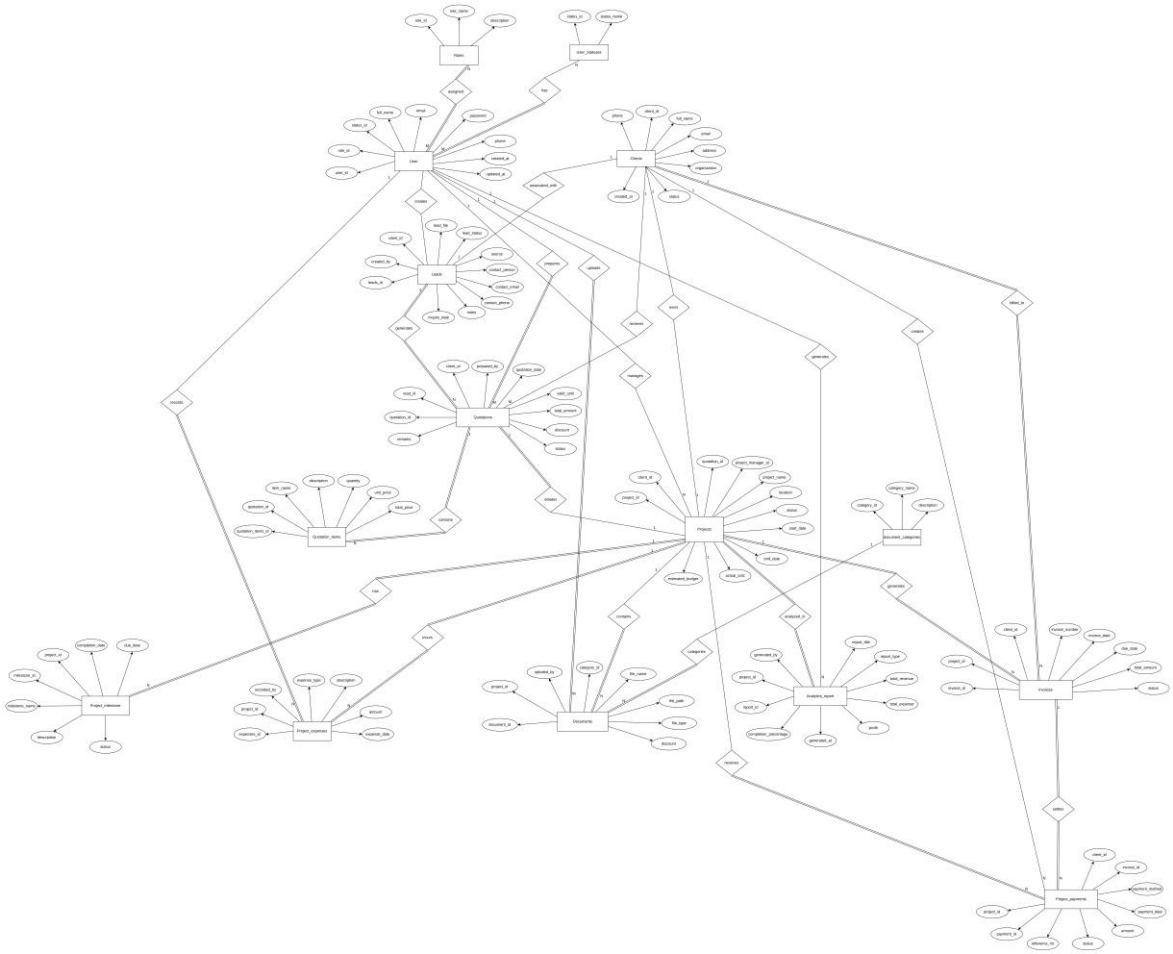


Figure 2: ER Diagram

This ER Diagram contains the **Entities** and **Relationship** between the Entities and **Cardinality Ratio** is contained in the ER Diagram.

2.1.1 Entity Relationship Diagram Description

The Entity Relationship (ER) diagram represents the logical structure of the **ConstructPro** database. It illustrates the entities, their attributes, and the relationships between them. The system is designed to manage clients, leads, quotations, projects, payments, documents, and analytics reports efficiently.

The **Users** entity stores information about system users such as administrators, managers, or employees. Each user is assigned a **Role** that defines their responsibilities and a **User Status** that indicates whether the user account is active or inactive.

The **Clients** entity maintains information about customers or organizations that request services. A client can create multiple **Leads**, which represent potential business opportunities. Leads may later be converted into **Quotations**, which contain detailed pricing information for services or products offered to the client.

Each quotation consists of multiple **Quotation Items**, representing individual services or products with their respective quantities and prices. When a quotation is approved, it may be converted into a **Project**. Projects store details such as project name, location, start date, end date, and budget information.

Projects are managed by a **Project Manager**, who is a user in the system. Each project can have several **Milestones** to track the progress of the work. The system also records **Project Expenses** to monitor operational costs and maintain financial transparency.

For billing purposes, the system generates **Invoices** for clients based on project progress. Clients can make **Project Payments**, which may be associated with specific invoices.

The system also supports document management through the **Documents** entity, where files related to projects are stored. Documents are categorized using **Document Categories** to maintain organization and easy retrieval.

Finally, the system provides analytical insights through **Analytics Reports**, which summarize project performance metrics such as total revenue, expenses, profit, and completion percentage.

Overall, the ER diagram demonstrates how different components of the system interact to support project lifecycle management, financial tracking, and document organization.

2.2 Normalization explanation

Database normalization is the process of organizing a database structure to reduce redundancy and improve data integrity. The ConstructPro ERP database design follows the standard normalization principles including **First Normal Form (1NF), Second Normal**

Form (2NF), and Third Normal Form (3NF) to ensure a well-structured relational database.

2.2.1 First Normal Form (1NF)

A table is said to be in **First Normal Form** if:

- Each table has a **primary key**.
- Each field contains **atomic values** (no repeating groups or arrays).
- Each column contains **data of a single type**.
- Each record is uniquely identifiable.

In the ConstructPro ERP data model, every entity satisfies 1NF

Examples:

Role Table

role_id (PK)	role_name	description

Table 1: Role Table

- Each role has a unique **role_id**.
- All attributes store single values.

User Table

user_id (PK)	full_name	email	role_id	status

Table 2: User Table

- Each user is uniquely identified by **user_id**.
- Role information is stored as a **foreign key** referencing the Role table instead of storing repeated role details.

Lead Table

lead_id (PK)	customer_name	phone	email	assigned_to

Table 3: Lead Table

- Each lead record contains atomic fields.
- No multi-valued attributes exist.

All other tables such as **Customer, Project, Task, Payment, Invoice, Expense, and Document** also follow the same structure.

Therefore, the database satisfies **First Normal Form (1NF)**.

2.2.2 Second Normal Form (2NF)

A table is in **Second Normal Form** if:

- It is already in **1NF**.
- All **non-key attributes depend entirely on the primary key**.
- There are **no partial dependencies**.

Since each table in the ConstructPro ERP system uses a **single-column primary key**, partial dependency problems do not occur.

Examples:

Quotation_Item Table

quotation_item_id (PK)	quotation_id (FK)	item_name	quantity	unit_price	amount

Table 4: Quotation_Item Table

- All attributes depend entirely on **quotation_item_id**.
- Item details are not dependent on only part of the key.

Task Table

task_id (PK)	project_id	milestone_id	assigned_to	task_name

Table 5: Task Table

- All task information depends fully on **task_id**.

Expense Table

expense_id (PK)	project_id	recorded_by	amount

Table 6: Expense Table

- Expense details depend only on **expense_id**.

Thus, all tables satisfy **Second Normal Form (2NF)**.

2.2.3 Third Normal Form (3NF)

A table is in **Third Normal Form** if:

- It is already in **2NF**.
- There are **no transitive dependencies** (non-key attributes depending on other non-key attributes).

In this design, related information is separated into different tables and connected through foreign keys.

Examples:

User and Role Separation

Instead of storing role names repeatedly in the User table:

User Table

user_id	full_name	role_id

Table 7: User Table

Role Table

role_id	role_name

Table 8: Role Table

This eliminates repeated role data and avoids update anomalies.

Customer and Lead Separation

Lead information is stored separately from customers:

Lead Table

lead_id	customer_name	phone

Table 9: Lead Table

Customer Table

customer_id	lead_id	full_name

Table 10: Customer Table

This structure allows a lead to be converted into a customer while maintaining historical lead data.

Project and Quotation Relationship

Projects reference quotations rather than duplicating quotation information.

Project Table

project_id	quotation_id

Table 11: Project Table

Quotation Table

quotation_id	quotation_date	total_amount

Table 12: Quotation Table

This ensures quotation details are stored only once.

Task and Milestone Relationship

Tasks reference milestones instead of duplicating milestone details.

Milestone Table

milestone_id	project_id

Table 13: Milestone Table

Task Table

task_id	milestone_id

Table 14: Task Table

This avoids storing milestone information repeatedly across tasks.

Elimination of Data Redundancy

The normalized structure eliminates unnecessary duplication of information.

Example:

Problem Without Normalization	Solution in Current Design
Repeating role names for every user	Separate Role table
Repeating customer details in projects	Customer table referenced by project
Repeating quotation items in quotations	Quotation_Item table
Repeating tasks in projects	Task table

Table 15: Elimination of Data Redundancy

Chapter 3 Class Diagram

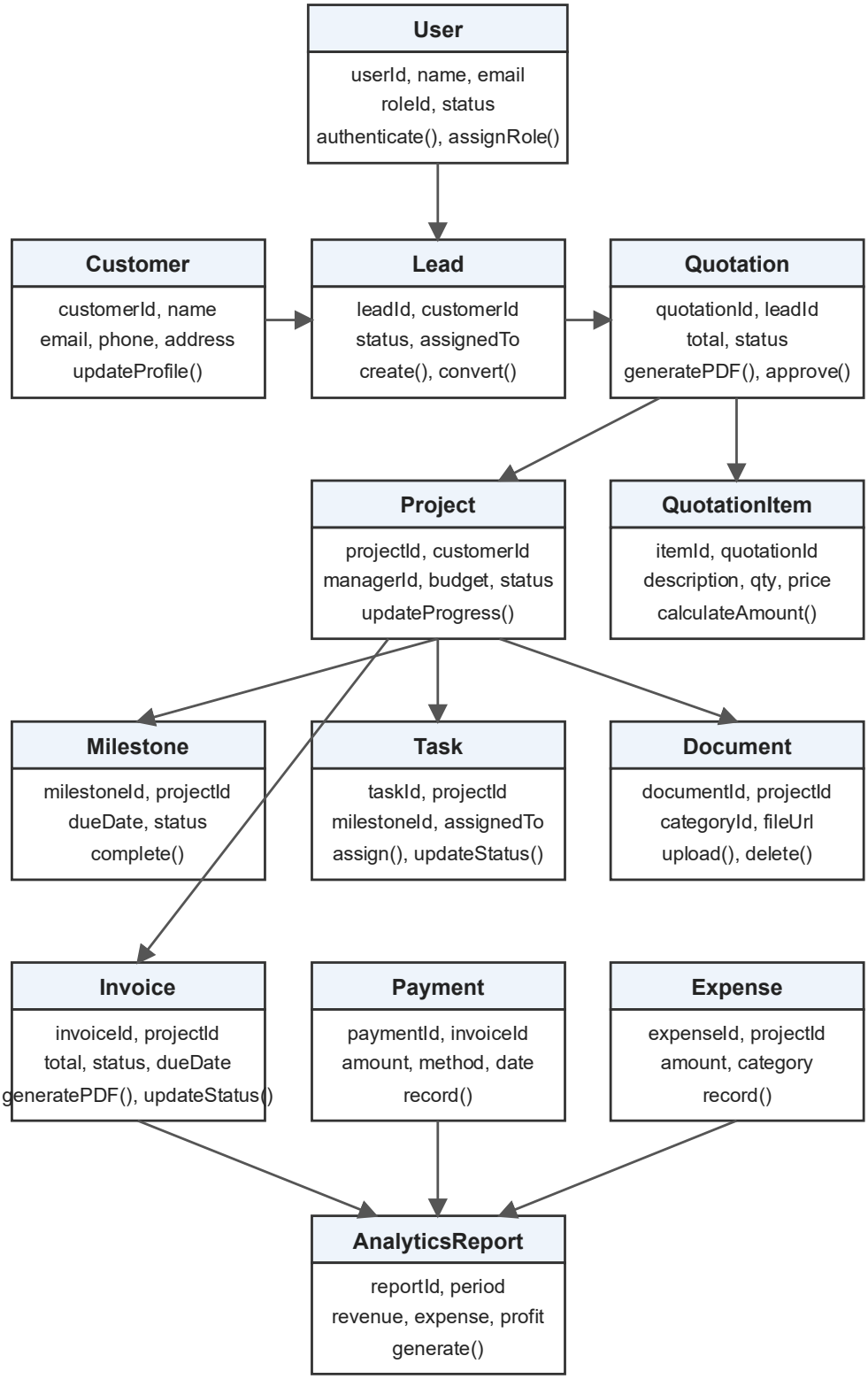


Figure 3: Class Diagram

3.1 Attributes

Attributes define the core data maintained by each implemented ConstructPro class.

User

userId, fullName, email, passwordHash, roleId, and account status.

Role

roleId, roleName, description, and associated permissions.

Customer

customerId, name, email, phone number, address, and organization details.

Lead

leadId, customer details, requirements, status, assigned user, and creation date.

Quotation

quotationId, leadId, customerId, status, subtotal, total amount, validity period, and generated PDF reference.

QuotationItem

quotationItemId, quotationId, description, quantity, unit price, and amount.

Project

projectId, customerId, quotationId, project manager, name, location, dates, budget, status, and progress.

Milestone

milestoneId, projectId, title, due date, completion date, and status.

Task

taskId, projectId, milestoneId, assigned user, task name, priority, due date, and status.

Invoice

invoiceId, projectId, customerId, total amount, due date, payment status, and PDF reference.

Payment

paymentId, invoiceId, amount, payment date, method, reference, and recorded user.

Expense

expenseId, projectId, amount, category, description, date, and recorded user.

Document

documentId, projectId, categoryId, file name, file URL, MIME type, and uploaded user.

AnalyticsReport

reportId, reporting period, revenue, expenses, profit, completion metrics, and generated date.

3.2 Methods

Methods represent the principal operations supported by the implemented domain classes.

User and Role

authenticate(), createUser(), updateUser(), assignRole(), and validatePermission().

Customer and Lead

createCustomer(), updateCustomer(), createLead(), assignLead(), updateLeadStatus(), and convertLead().

Quotation and QuotationItem

createQuotation(), addItem(), calculateTotal(), updateStatus(), approveQuotation(), and generatePDF().

Project, Milestone, and Task

createProject(), updateProject(), updateProgress(), createMilestone(), completeMilestone(), createTask(), assignTask(), and updateTaskStatus().

Invoice and Payment

createInvoice(), generatePDF(), recordPayment(), calculateOutstandingBalance(), and updatePaymentStatus().

Expense and Document

recordExpense(), updateExpense(), uploadDocument(), categorizeDocument(), retrieveDocument(), and deleteDocument().

AnalyticsReport

aggregateMetrics(), calculateRevenue(), calculateProfit(), calculateCompletionRate(), and generateReport().

3.3 Relationships

Relationships define how the implemented classes collaborate across ConstructPro workflows.

A Role can be assigned to many Users, while each User has one active role.

A Customer can have many Leads, Quotations, Projects, and Invoices.

A Lead may produce one or more Quotations, and each Quotation contains one or more QuotationItems.

An approved Quotation may be converted into a Project.

A Project is managed by a User and can contain many Milestones, Tasks, Expenses, Documents, and Invoices.

A Milestone can contain many Tasks, and each Task may be assigned to a User.

An Invoice can have one or more Payments, and its status is updated from the recorded payment total.

AnalyticsReport aggregates project and financial records to produce business performance metrics.

3.4 Visibility

Visibility controls how class data and operations are accessed.

Identifiers and internal state are treated as private implementation details. Public service operations expose validated actions such as creating records, updating status, recording payments, generating PDFs, managing documents, and producing analytics.

Access to these operations is restricted through authentication and role-based access control.

3.5 Multiplicity

Multiplicity defines the number of instances participating in each relationship.

Role to User: one-to-many.

Customer to Lead, Quotation, Project, and Invoice: one-to-many.

Lead to Quotation: one-to-many; Quotation to QuotationItem: one-to-many.

Quotation to Project: zero-or-one project for an approved quotation.

Project to Milestone, Task, Expense, Document, and Invoice: one-to-many.

Milestone to Task: one-to-many; User to assigned Task: one-to-many.

Invoice to Payment: one-to-many.

These multiplicities reflect the implemented ConstructPro ERP domain model.

Chapter 4 Sequence Diagrams

4.1 Authentication & Login Sequence Diagram

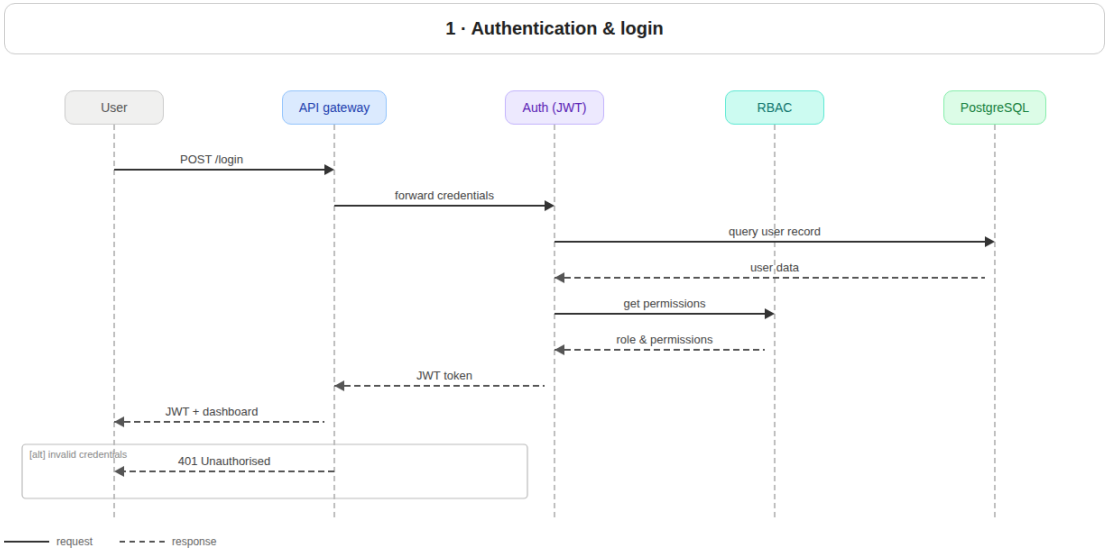


Figure 4: Authentication & Login Sequence Diagram

4.2 Lead Capture & Quotation Generation Sequence Diagram

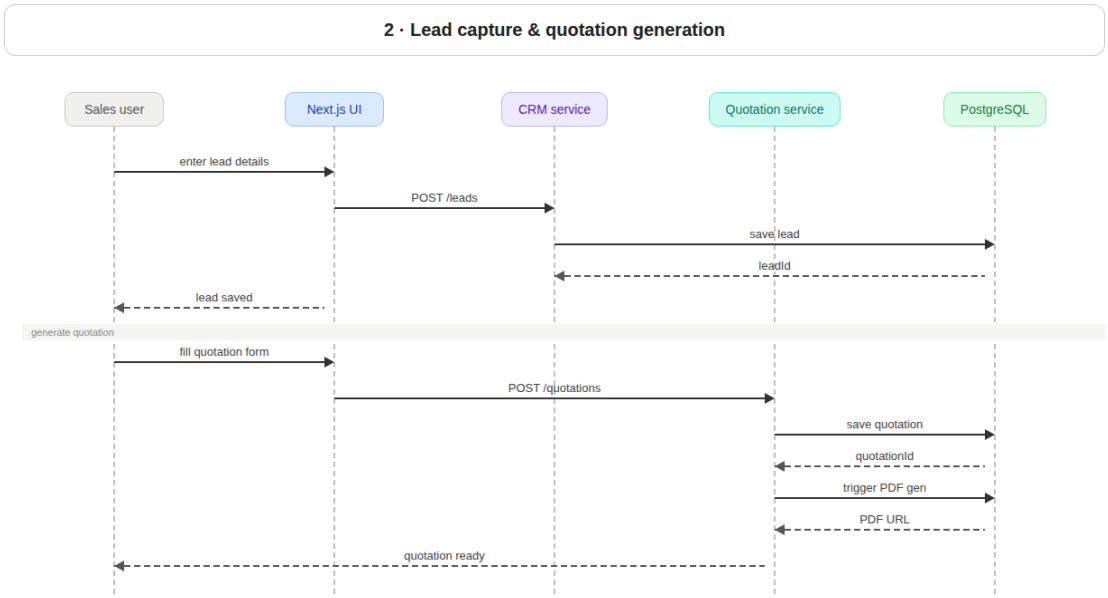


Figure 5: Lead Capture & Quotation Generation Sequence Diagram

4.3 Quotation Approval & Project Creation Sequence Diagram

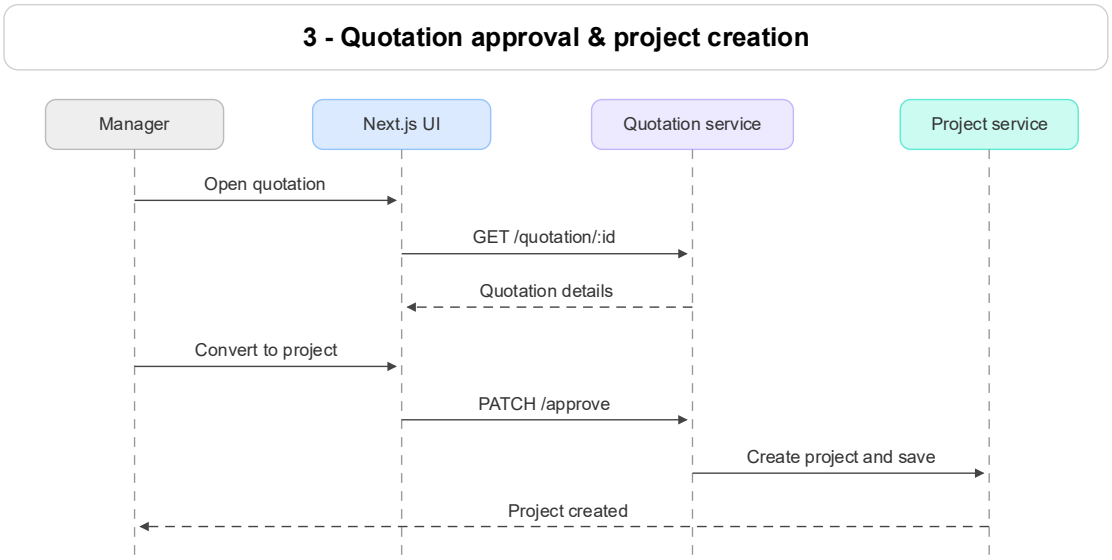


Figure 6: Quotation Approval & Project Creation Sequence Diagram

4.4 Payment Recording & Invoice Management Sequence Diagram

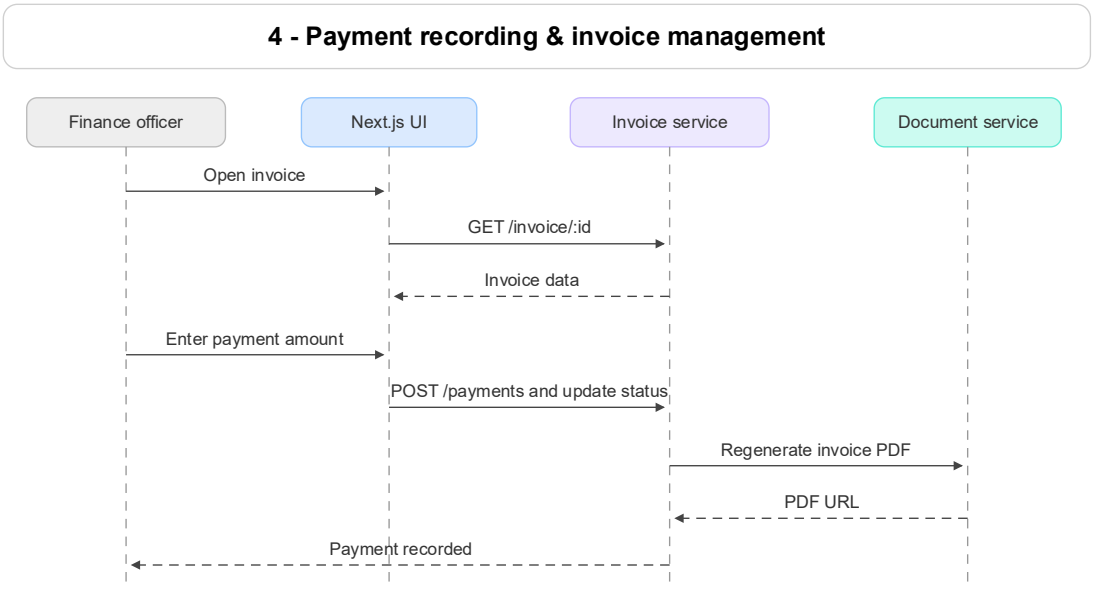


Figure 7: Payment Recording & Invoice Management Sequence Diagram

4.5 AI Rist Prediction Sequence Diagram

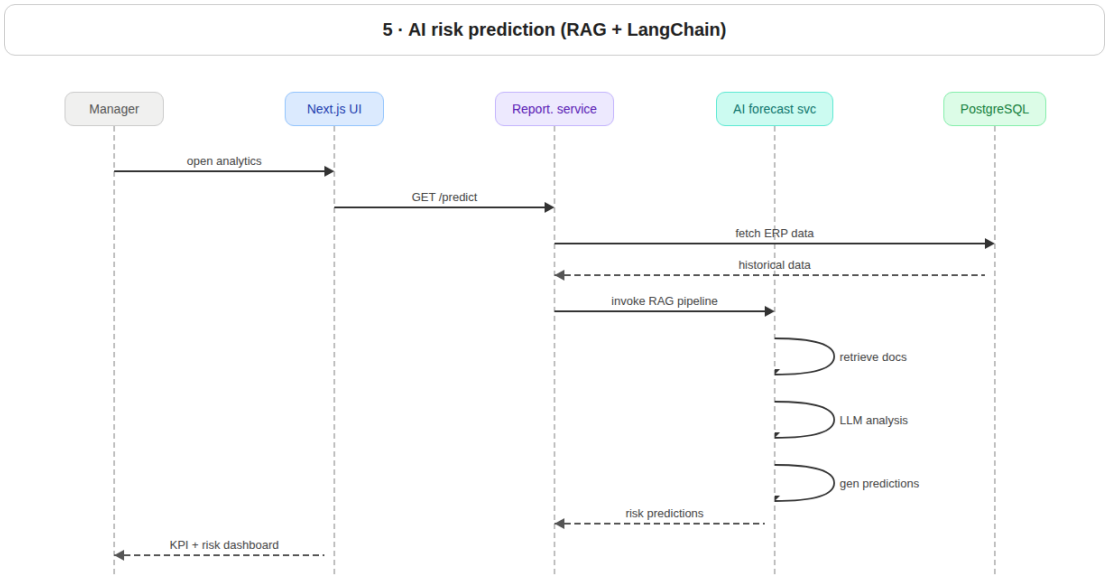


Figure 8: AI Rist Prediction Sequence Diagram

Chapter 5 UI/UX Design

5.1 Wireframes or screen mock-ups for key user interfaces

The following screen mock-ups illustrate the key user interfaces of the ConstructPro ERP system, developed using Figma [7]. Each interface reflects the role-based access control structure and the core functional modules of the system. In addition to the screens presented below, the complete design file includes supplementary interfaces such as authentication screens, typography specifications, color palettes, and component libraries, which are accessible via the Figma prototype referenced in [7].

5.1.1 Dashboard UI

The Dashboard provides a consolidated overview of active projects, revenue metrics, outstanding payments, and recent system activity.

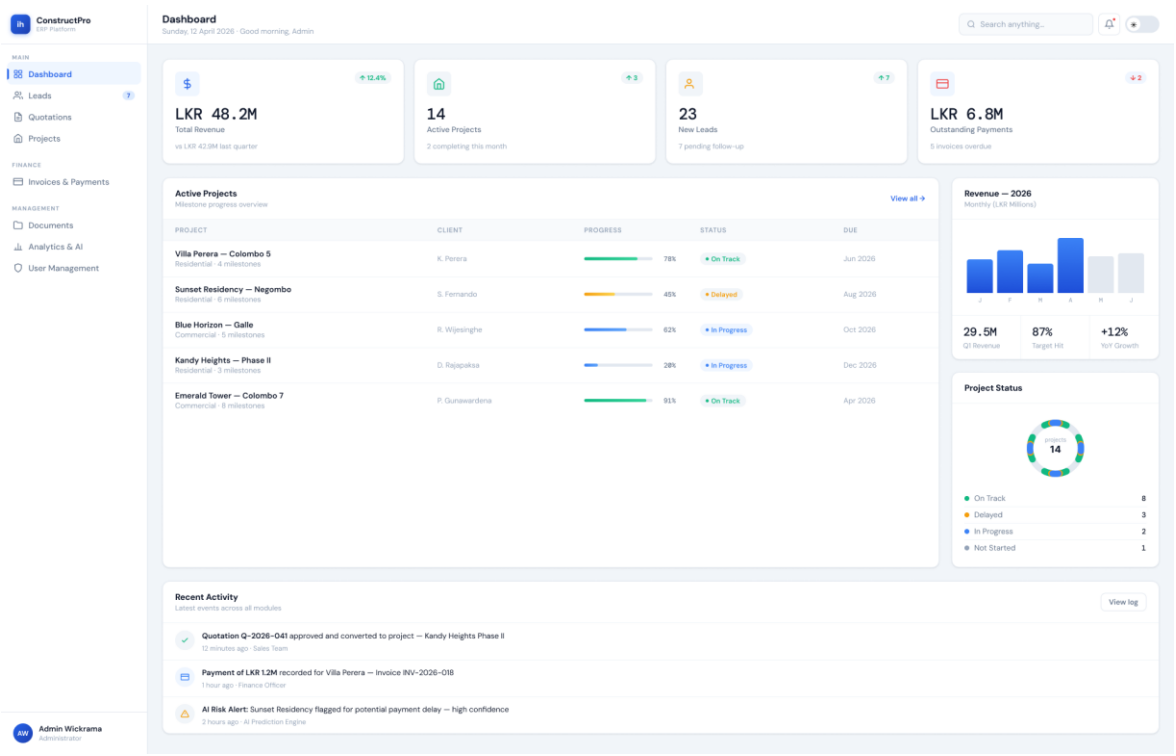


Figure 9: Dashboard UI

5.1.2 Lead Management UI

The Lead Management interface allows the sales team to capture, filter, and track client leads with detailed status indicators.

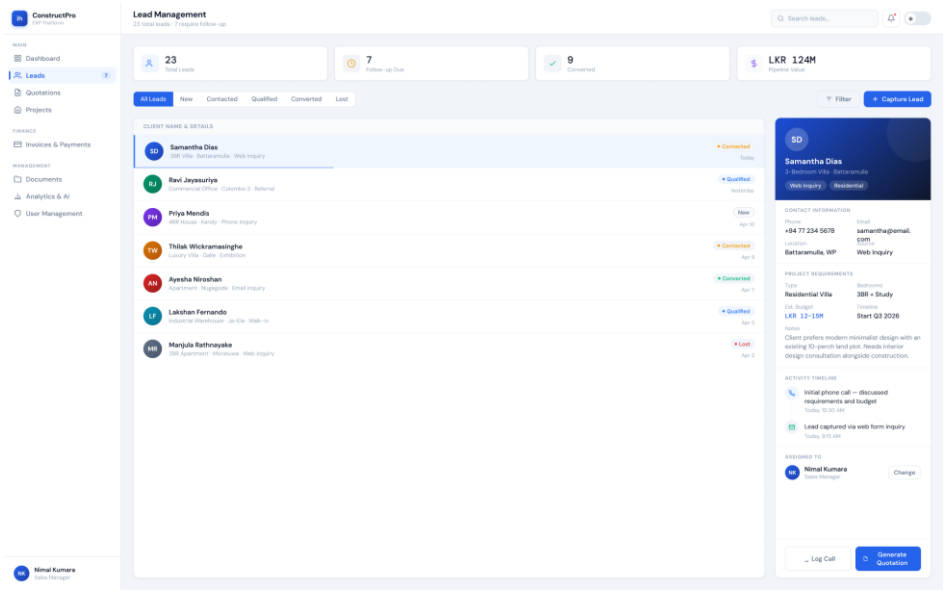


Figure 10: Lead Management UI

5.1.3 Quotations UI

The Quotations interface enables the creation, review, and approval of client quotations with line-item breakdowns and status tracking.

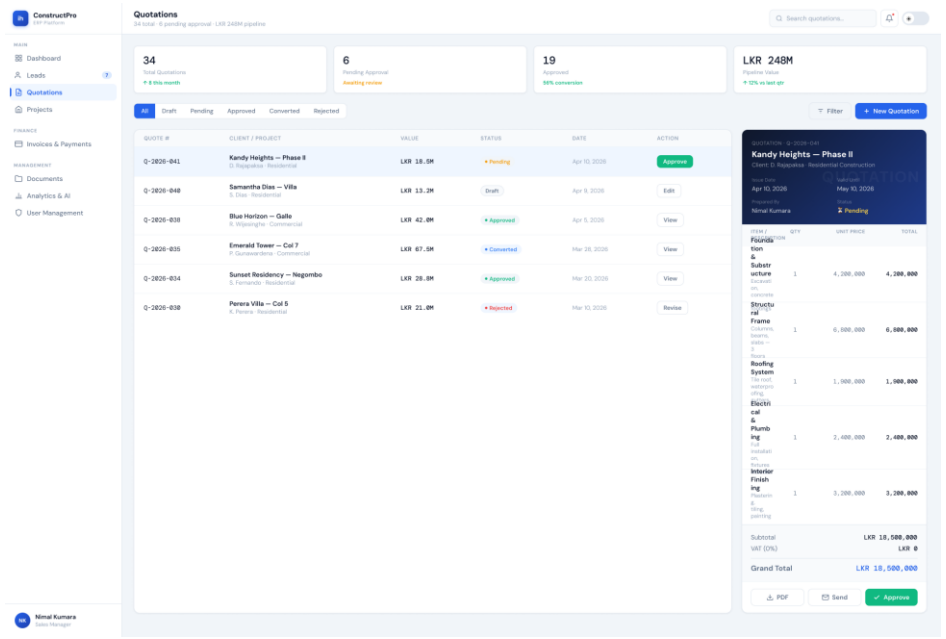


Figure 11: Quotations UI

5.1.4 Projects UI

The Projects interface displays active project details, milestone progress, and task completion percentages for project managers.

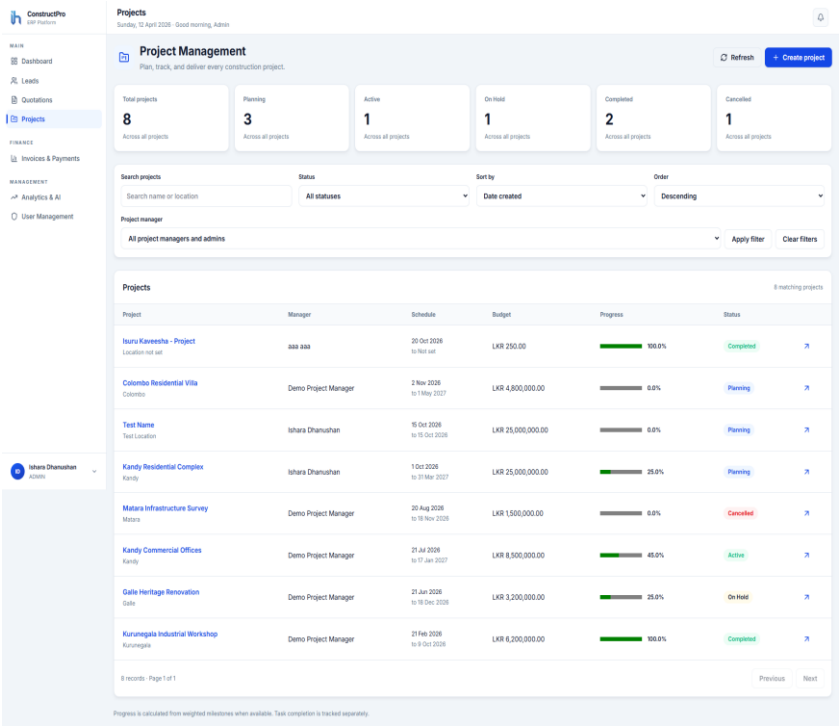


Figure 12: Projects UI

5.1.5 Invoices & Payments UI

The Projects interface displays active project details, milestone progress, and task completion percentages for project managers.

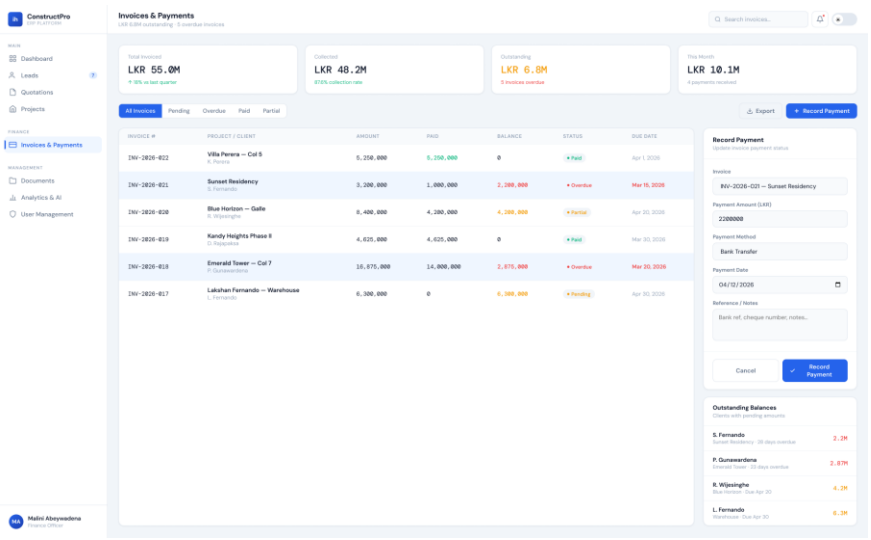


Figure 13: Invoices & Payments UI

5.1.6 Document Management UI

The Document Management interface supports file uploads, categorization, and retrieval of project-related documents.

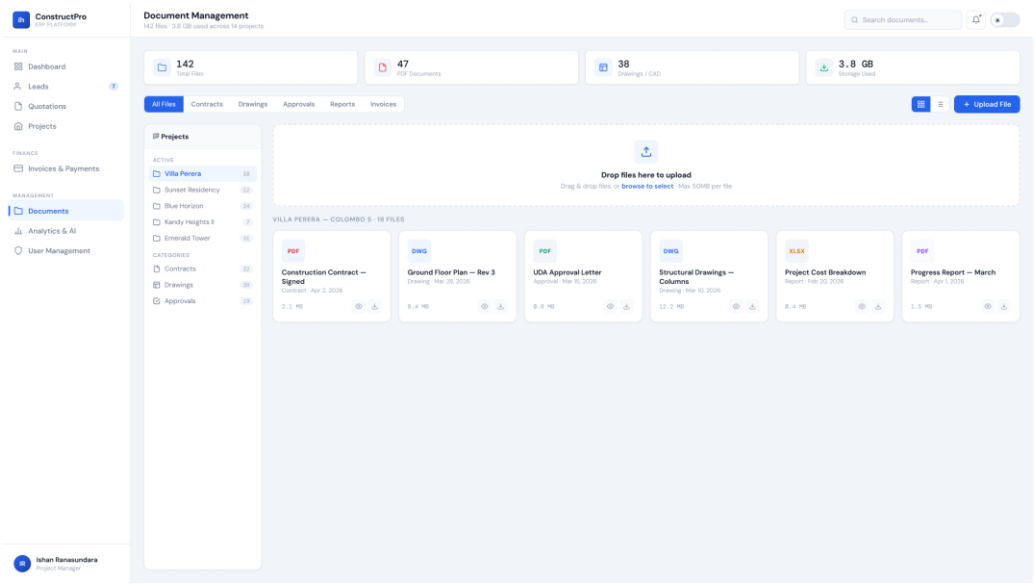


Figure 14: Document Management UI

5.1.7 Analytics & AI Risk Prediction UI

The Analytics interface presents revenue trends, performance metrics, and AI-generated project risk scores.

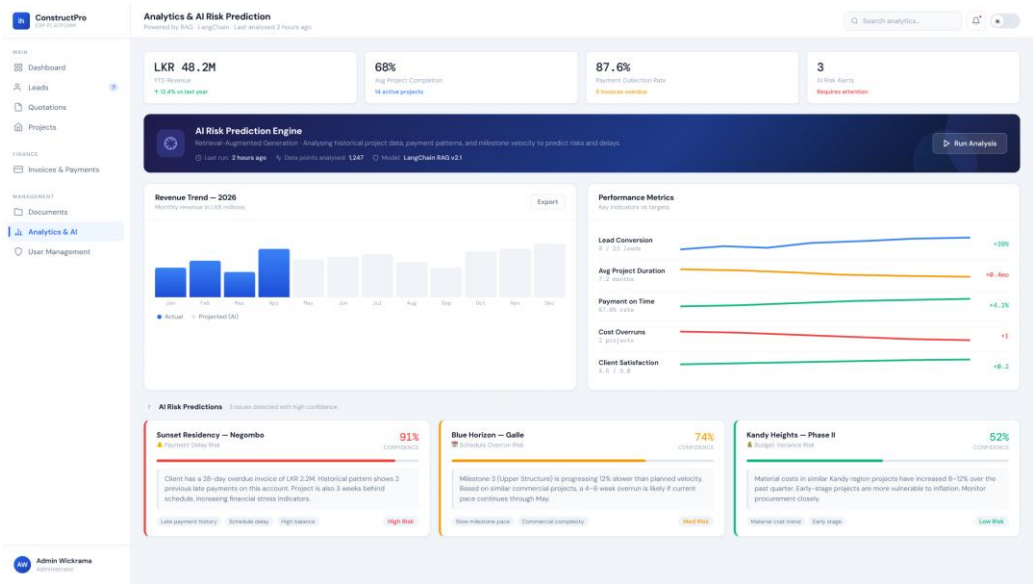


Figure 15: Analytics & AI Risk Prediction UI

5.1.8 User Management UI

The User Management interface allows administrators to create users, assign roles, and configure module-level access permissions.

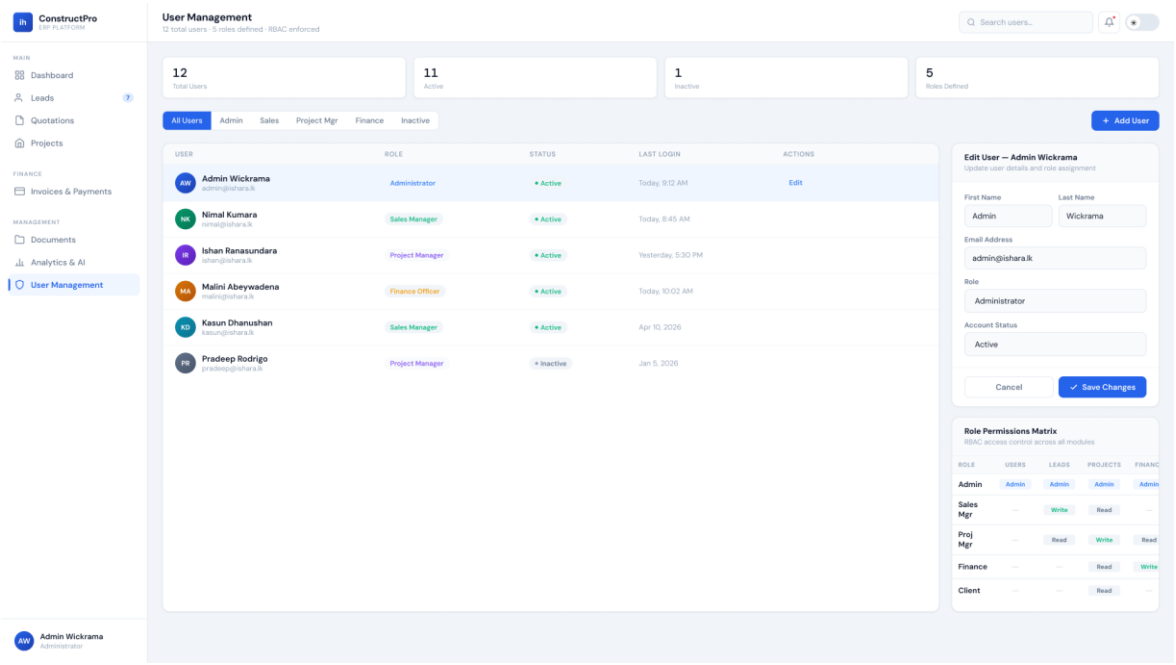


Figure 16: User Management

5.2 User flow / navigation diagrams

5.2.1 Role-based user flow diagram

This diagram illustrates how different user roles navigate the system following authentication, with each role directed to its permitted modules based on JWT-validated RBAC rules.

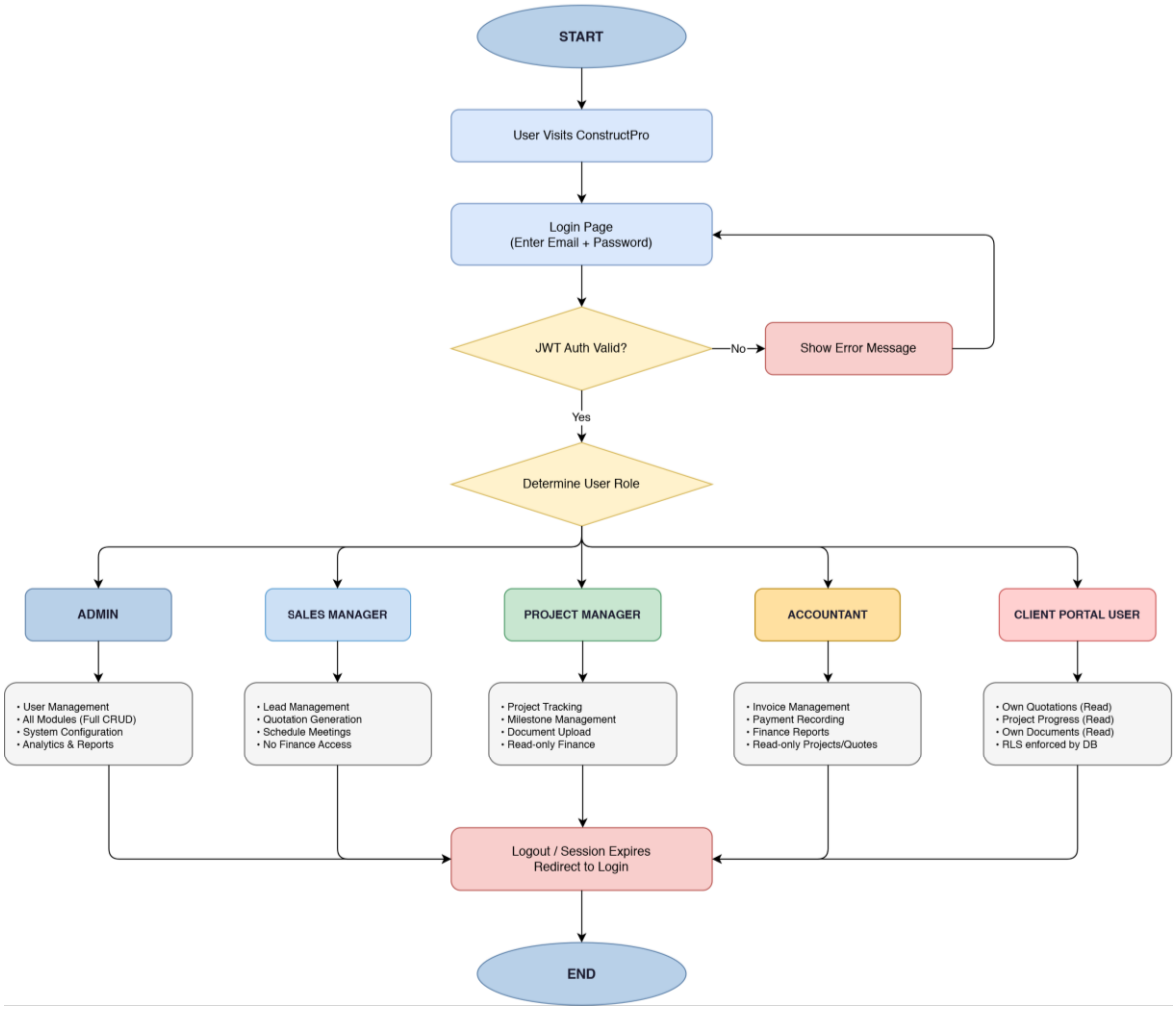


Figure 17: Role-based user flow diagram

5.2.2 Lead-quotation-project flow diagram

This diagram depicts the end-to-end workflow from lead capture through quotation generation to project creation, involving the sales team, management, and the client.

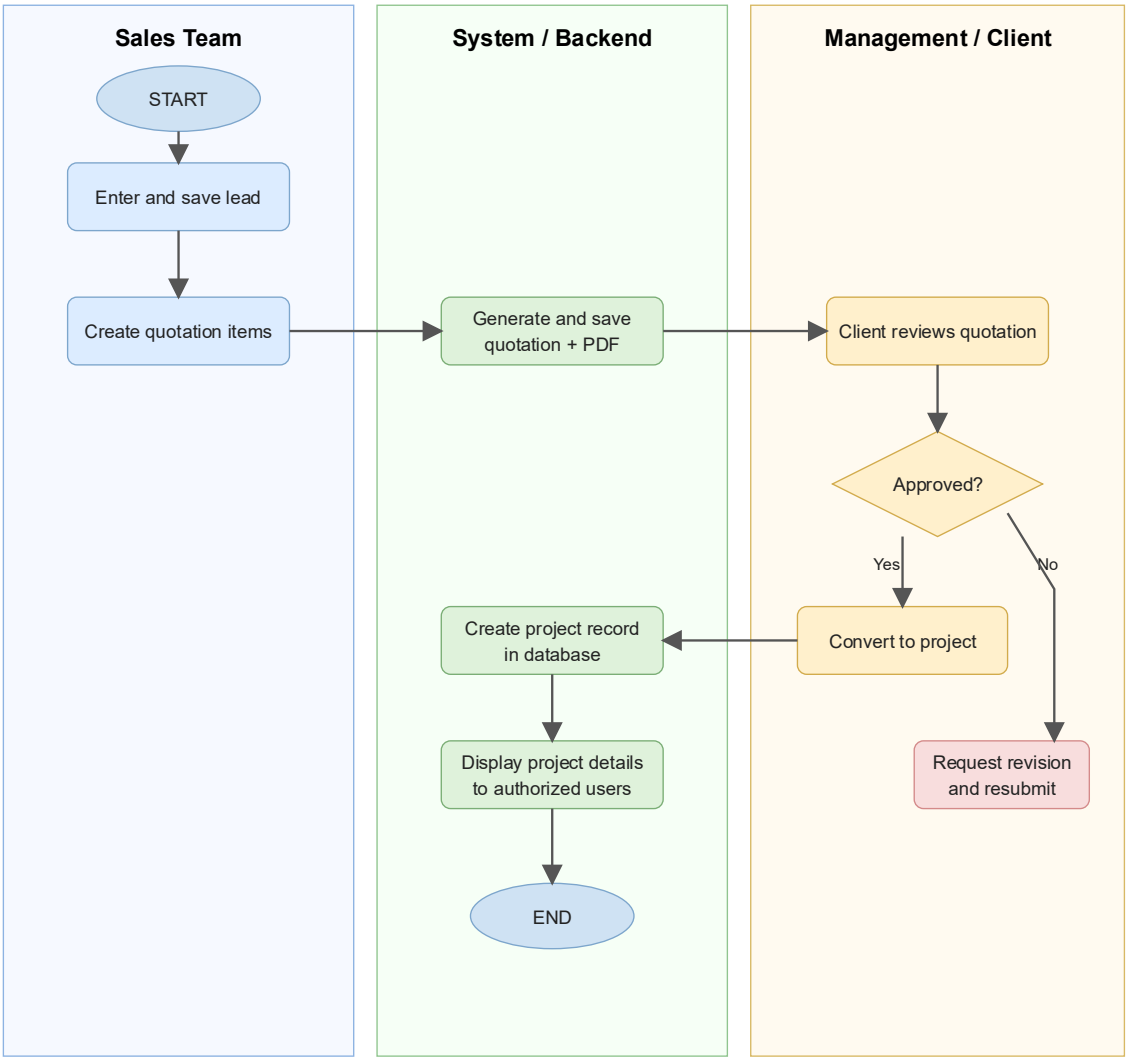


Figure 18: Lead-quotation-project flow diagram

5.2.3 Project management flow

This diagram outlines the actions available to the project manager within an active project, including milestone management, document uploads, and meeting scheduling via Google Calendar.

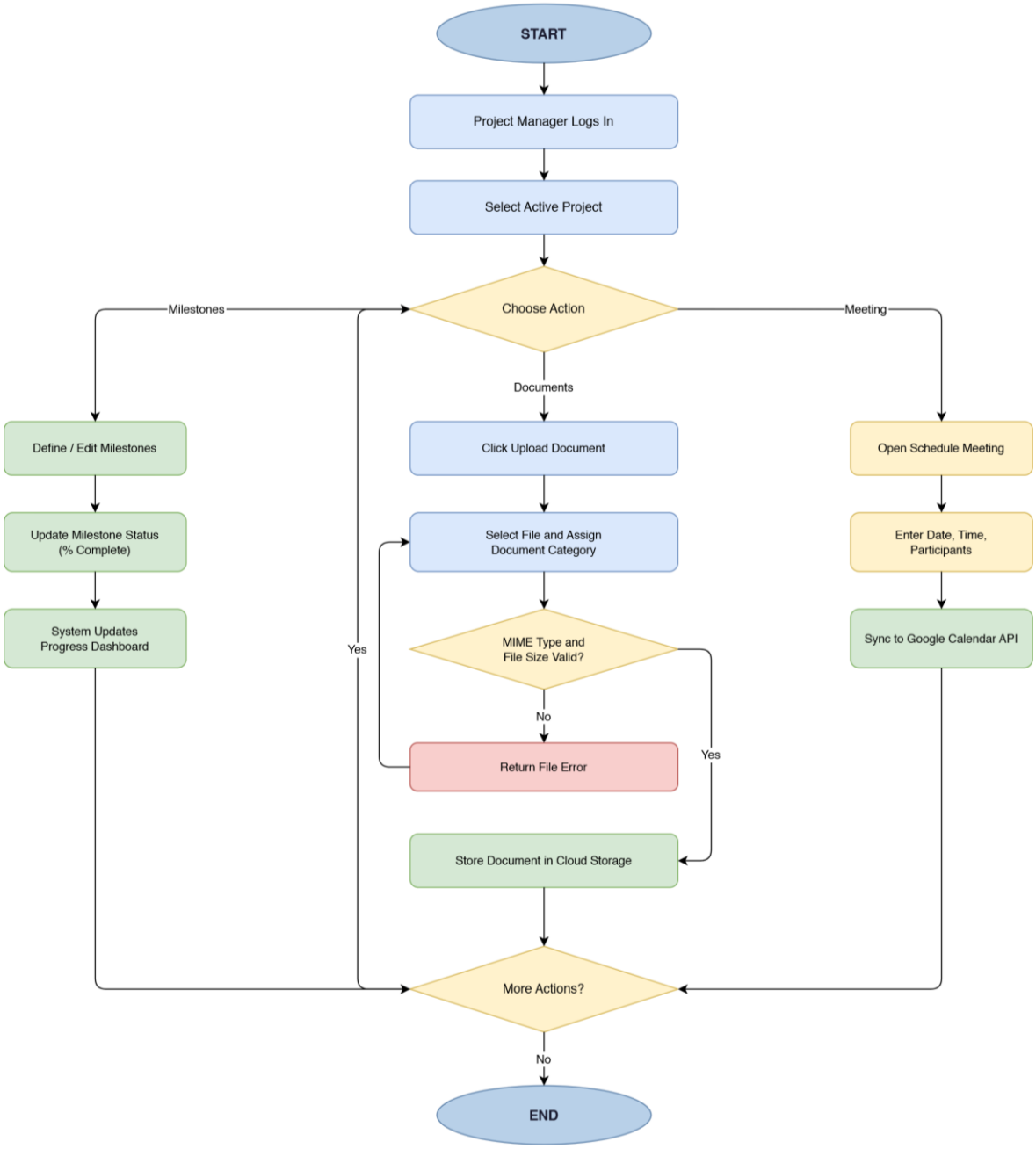


Figure 19: Project management flow

5.2.4 Finance and payment management flow

This diagram illustrates the accountant's workflow for creating invoices, recording payments, and generating financial reports within the Finance module.



Figure 20: Finance and payment management flow

5.2.5 Analytics and AI risk prediction flow

This diagram describes the process by which the system retrieves historical ERP data, invokes the LangChain RAG pipeline, and generates revenue trend predictions and project risk scores.

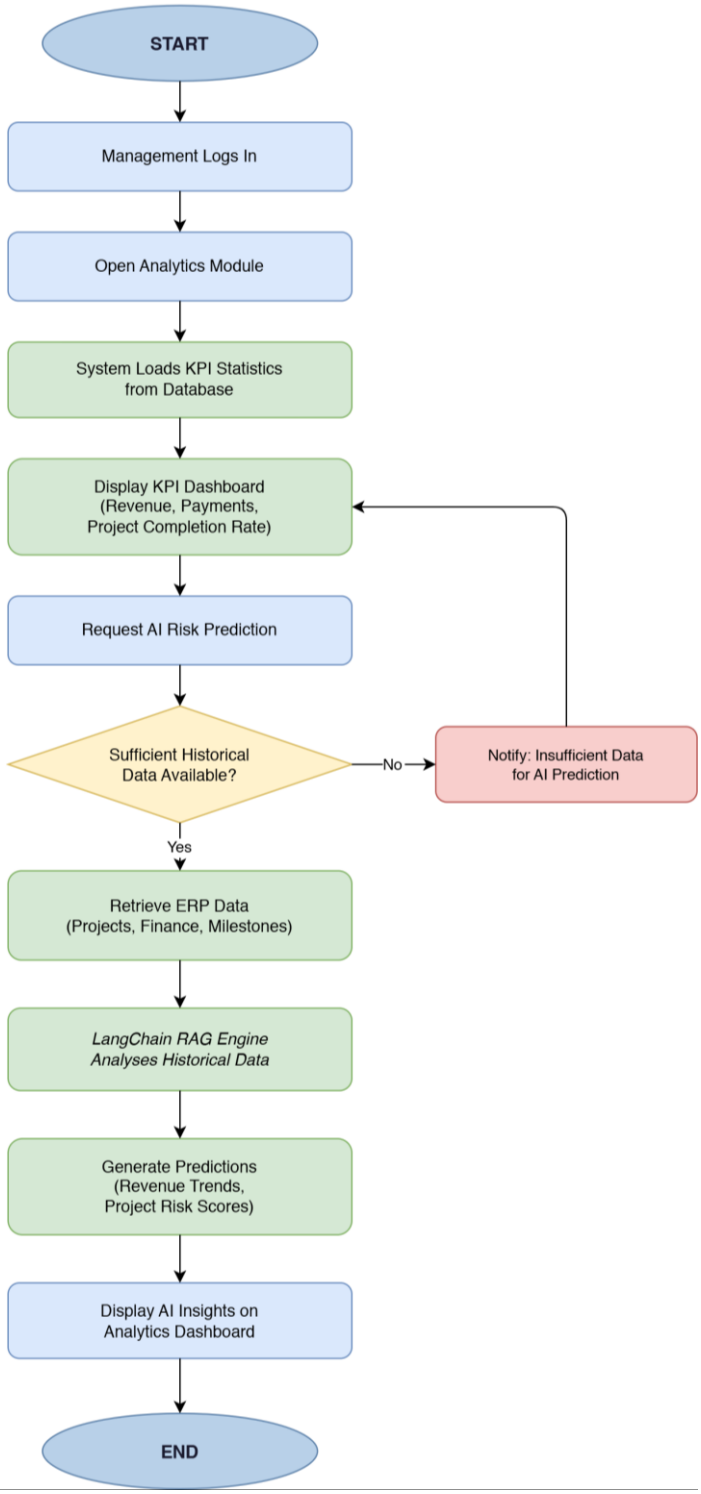


Figure 21: Analytics and AI risk prediction flow

5.2.6 Document management flow

This diagram presents the document lifecycle within the system, covering file upload with MIME type validation, cloud storage, category-based browsing, and deletion workflows.

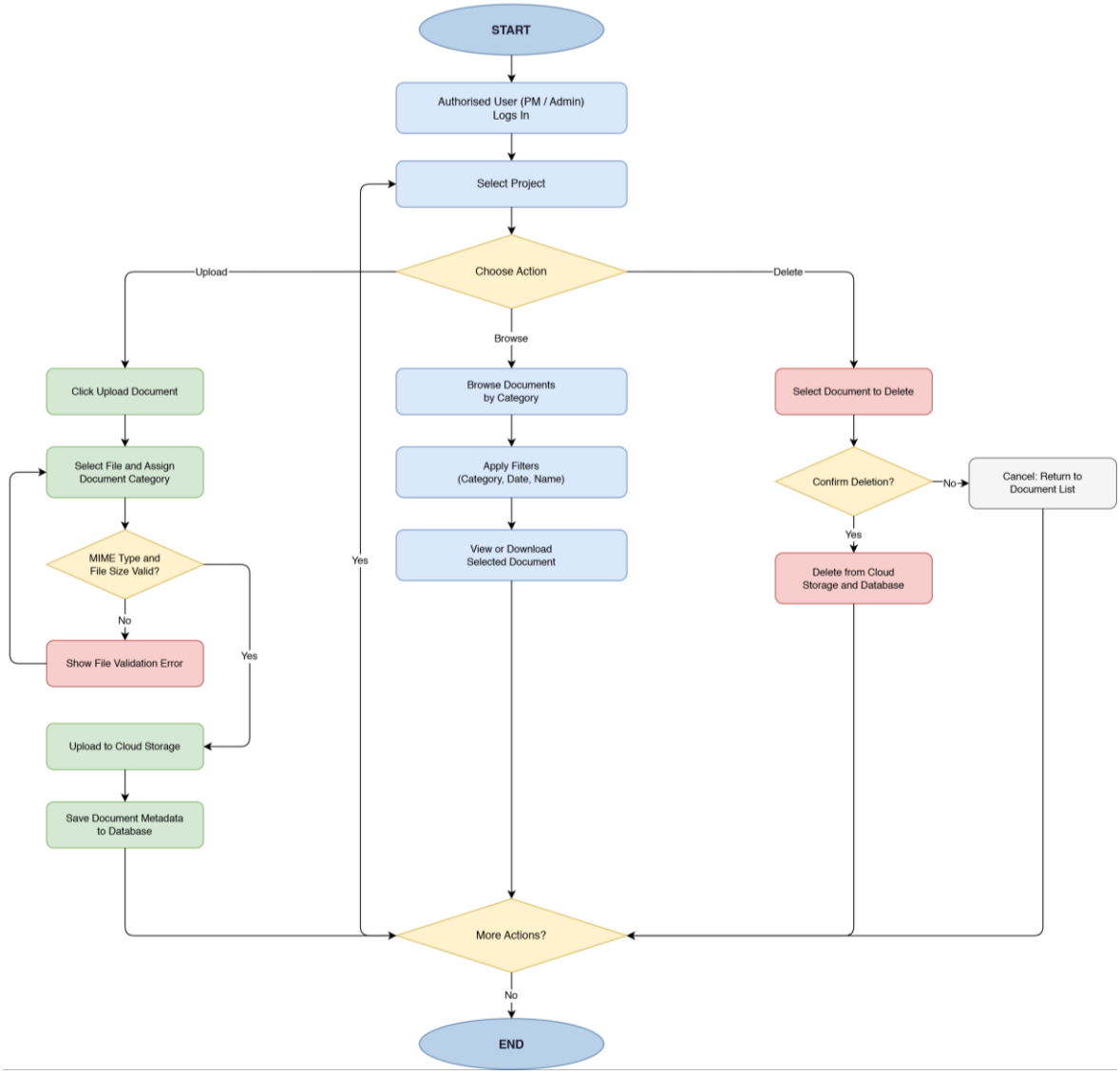


Figure 22: Document management flow

5.3 Accessibility considerations

Accessibility is a fundamental aspect of the ConstructPro ERP system design, ensuring that the platform can be effectively used by all users regardless of their abilities or assistive technology requirements. The system is designed to conform to the **Web Content Accessibility Guidelines (WCAG) 2.1 at Level AA**, as established by the World Wide Web Consortium (W3C). These guidelines are organized under four core principles - **Perceivable, Operable, Understandable, and Robust (POUR)** - each of which addresses a distinct dimension of accessible design. Conformance to WCAG 2.1 Level AA is considered the industry-standard baseline for enterprise web applications and is applied across all user-facing modules of the system, including the Dashboard, Lead Management, Quotation, Project Management, Finance, Document Management, and Client Portal interfaces. The following considerations are implemented in accordance with these principles.

5.3.1 Perceivable

- All non-text content, including dashboard charts, KPI visualizations, and AI risk prediction graphs, is accompanied by descriptive alternative text for screen reader compatibility.
- Color is not used as the sole means of conveying information. Project status indicators and risk level classifications are supplemented with textual labels alongside color coding.
- A minimum contrast ratio of **4.5:1** is maintained between foreground text and background across all modules; a ratio of **3:1** is applied for large text elements such as section headings.

5.3.2 Operable

- All interactive components including navigation menus, forms, modals, dropdowns, and action buttons, are fully operable through keyboard input alone.
- A logical tab order is maintained across all screens, consistent with the visual layout.
- Visible focus indicators are provided on all focusable elements.
- Time-sensitive operations, such as session expiry, provide users with sufficient warning and the option to extend time limits.

5.3.3 Understandable

- All form inputs across Lead Management, Quotation, Milestone, and Payment modules are associated with clearly labeled elements.
- Required fields are identified both visually and programmatically using `aria-required`.
- Descriptive error messages are displayed adjacent to the relevant input field and linked using `aria-describedby`, enabling screen readers to announce specific errors.
- Field instructions and hints are presented before the input field, not after.

5.3.4 Robust

- The Next.js frontend employs semantic HTML5 elements (<nav>, <main>, <section>, <table>, <button>) to ensure compatibility with assistive technologies.
- Dynamic components such as the AI risk prediction panel and live KPI dashboard updates use `aria-live` regions so that content changes are announced to screen readers without a page reload.
- All components are tested for compatibility with major screen readers including NVDA and browser-native accessibility tools.

5.3.5 Additional Considerations

- Generated PDF documents (quotations and invoices) include tagged headings and reading order metadata to ensure accessibility in PDF reader applications.
- The interface supports browser-level text scaling up to **200%** without loss of content or functionality (WCAG 2.1 SC 1.4.4).
- Role-restricted UI elements are communicated using `aria-disabled` with a descriptive tooltip, rather than being silently hidden from the interface.

Chapter 6 Security Design

6.1 Authentication mechanism

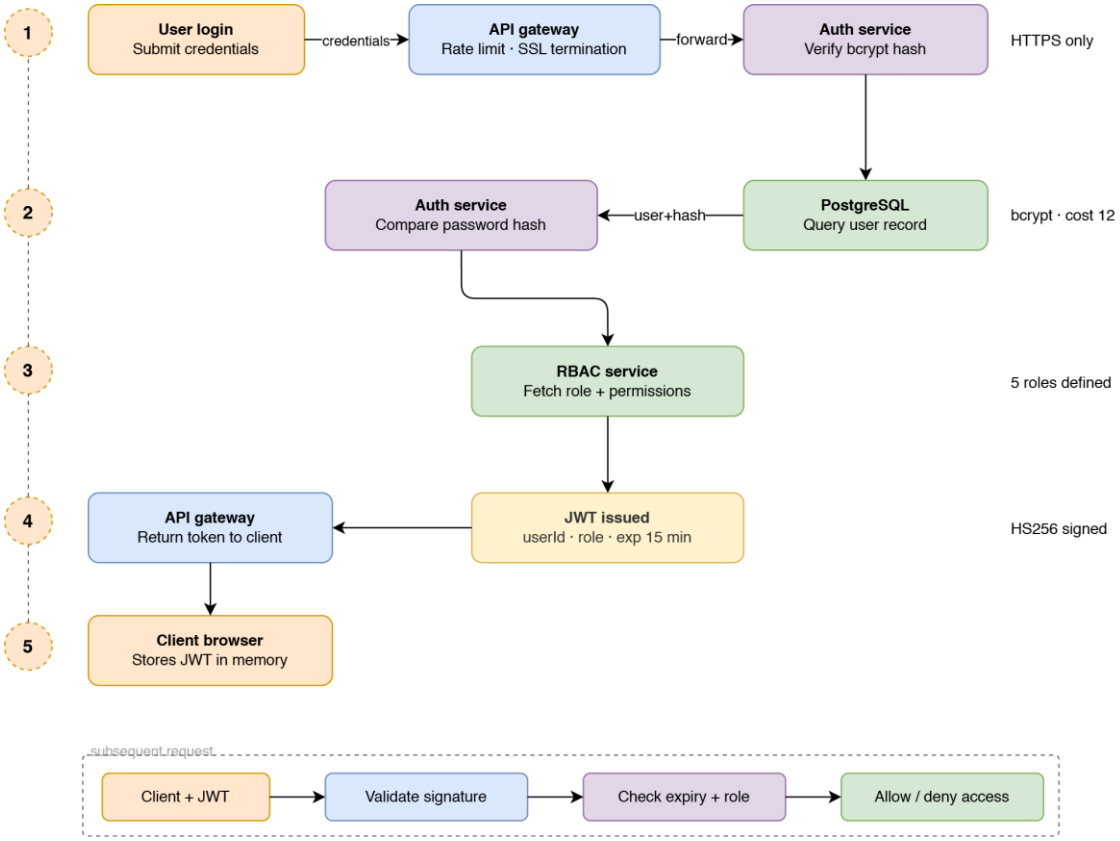


Figure 23: Authentication mechanism

ConstructPro uses JSON Web Token (JWT) based stateless authentication. When a user submits their credentials, the request travels over HTTPS to the API Gateway, which enforces rate limiting and handles SSL termination before forwarding to the Auth service. The Auth service queries **PostgreSQL** to retrieve the stored user record and verifies the submitted password against a bcrypt hash with a cost factor of 12. On successful verification, the Auth service calls the RBAC service to retrieve the user's role and associated permissions, then issues a signed JWT using HS256 with a 256-bit secret key.

The token payload contains the userId, role, and an expiry timestamp set to 15 minutes. A separate refresh token with a 7-day expiry is issued alongside it and stored as an HTTP-only cookie to prevent JavaScript access. For every subsequent API request, the client sends the access token in the Authorization: Bearer header. The API Gateway validates the token signature and expiry before the request reaches any backend service, no service handles unauthenticated requests. On logout, the refresh token is invalidated server-side via a token blocklist stored in **PostgreSQL**.

6.2 Authorization levels

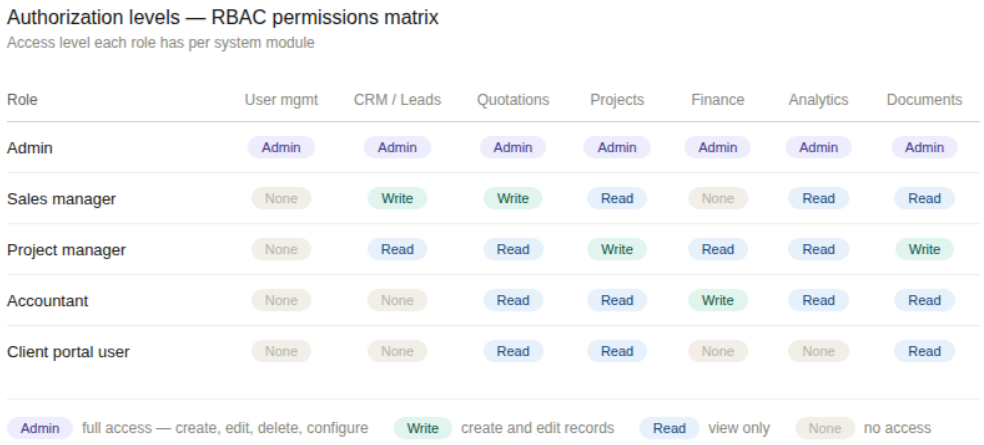


Figure 24: Authorization levels

ConstructPro implements Role-Based Access Control (RBAC) with five predefined roles: Admin, Sales Manager, Project Manager, Accountant, and Client Portal User. Permissions are enforced at the **NestJS Guards layer** after JWT validation, the RBAC guard checks the role embedded in the token against the required permission for the requested route before any service logic executes. [4]

The admin role has unrestricted access across all modules including user management, configuration, and all CRUD operations. The Sales Manager can create and edit leads and quotations but has no access to finance data, maintaining a clear separation between sales and financial visibility. The Project Manager has written access to projects and documents but can only read quotations and finance records enough visibility to do their job without modifying financial records. The accountant has written access exclusively to the Finance

module, with read-only access to quotations and projects for context. The Client Portal User has the most restricted access read-only visibility into their own quotations, project progress, and documents, enforced through **PostgreSQL row-level security (RLS) policies using** {clientId: req.user.id} so they cannot access other clients' data despite holding a Read permission.

6.3 Data protection approach

Data protection in ConstructPro is addressed across three layers. For data in transit, all communication between clients and the API Gateway is enforced over TLS 1.2 minimum (TLS 1.3 preferred), with HTTP requests automatically redirected to HTTPS. Internal service-to-service communication within the backend also operates over encrypted channels. For data at rest, **Neon PostgreSQL provides volume-level encryption using AES-256**, covering all tables including client records, quotations, payment data, and project documents. Cloud File Storage uses AES-256 Server-Side Encryption for all uploaded files such as contracts and drawings. [5]

For data access control, each backend microservice connects to **PostgreSQL** using a dedicated database user with the minimum required privileges, the CRM service cannot query the Finance tables, for example. Sensitive fields such as payment amounts, client contact details, and contract values are treated as confidential and are never returned in API list responses only in single-record detail endpoints after authorization checks pass.

6.4 Input validation and sanitization strategy

All incoming request data is validated and sanitized at two layers before reaching any service logic. At the schema validation layer, every API endpoint defines an expected request shape using **Zod**, integrated with NestJS validation pipes. Requests that do not match the schema, missing required fields, wrong data types, strings where numbers are expected, are rejected with a 400 Bad Request before touching the database. This applies to all request bodies, query parameters, and route parameters.

At the sanitization layer, all string inputs are processed to prevent **SQL injection attacks**, which are the primary injection threat for PostgreSQL. The system uses **NestJS's built-in ValidationPipe combined with parameterized queries via the ORM (TypeORM or Prisma)**, ensuring that user-supplied values are never interpolated directly into SQL statements. File uploads handled by the File Service are validated server-side for MIME type (not just file extension, which can be spoofed) and enforced size limits, regardless of any client-side validation. Error messages returned to clients are generic and never expose internal field names, stack traces, or database structure. All validation errors are logged internally with full context for debugging without surfacing details externally. [4]

6.5 Encryption standards for data at rest and in transit

For data in transit, ConstructPro enforces TLS 1.2 as the minimum protocol version across all endpoints, with TLS 1.3 as the preferred version. Certificates are provisioned and auto-renewed via Let's Encrypt or the cloud provider's Certificate Manager. Cipher suites are restricted to forward-secrecy variants (ECDHE key exchange) so that a future compromise of the server's private key cannot be used to decrypt past sessions. HTTP Strict Transport Security (HSTS) headers are set with a minimum max-age of one year to prevent protocol downgrade attacks.

For data at rest, **Neon PostgreSQL applies AES-256 encryption at the storage layer**, so all data files, indexes, and write-ahead log (WAL) entries are encrypted on disk. Cloud File Storage applies AES-256 SSE at the object level, with server-managed keys rotated annually. User passwords are never stored in plaintext or as reversible encryption, they are hashed exclusively using bcrypt with a cost factor of 12, which is computationally expensive enough to resist brute-force attacks while remaining practical for login latency. JWT tokens are signed using HS256 (HMAC-SHA256) with a secret key of at least 256 bits, stored as an environment variable and never committed to source control. API keys and environment secrets across all services are managed through the cloud provider's secrets manager rather than hardcoded in configuration files.

Chapter 7 Deployment Design

7.1 Hosting Plan

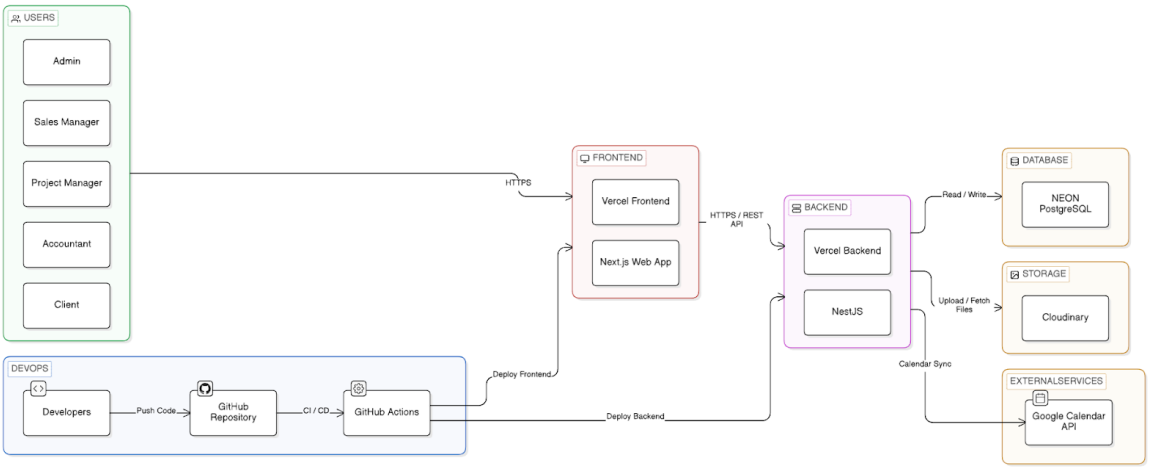


Figure 25: Hosting Plan

The ERP system will use a cloud-based deployment model centered on Vercel. The Next.js frontend will be deployed on Vercel for global delivery, SSL, CDN support, and easy preview deployments. The NestJS backend will also be deployed on Vercel using Node.js server-side functions, exposing REST API endpoints for authentication, CRM, quotations, invoices, projects, reporting, files, calendar integrations, and analytics workflows. Vercel supports NestJS deployments and Node.js server-side execution, making this feasible for the current phase of the project. [2] [3]

The application data will be stored in PostgreSQL hosted on Neon, which acts as the primary relational database for users, leads, customers, quotations, projects, milestones, tasks, invoices, payments, documents, expenses, and analytics records. Uploaded files and generated assets will be stored in Cloudinary, which provides upload APIs, storage, transformation, and delivery capabilities for media and raw files. This is suitable for quotation PDFs, invoice documents, project attachments, and client-uploaded files. [5]

The deployment workflow will be integrated with GitHub and GitHub Actions. Source code will be managed in GitHub repositories, and GitHub Actions will be used to run quality checks, builds, and controlled deployments to Vercel. Vercel also supports Git-based automatic deployments and preview URLs for non-production branches, which align well with development and staging workflows. [2]

7.2 Hardware Requirements

Because this solution is deployed mainly on managed cloud services, dedicated physical server hardware is not required for the current implementation. Instead, the project depends on managed runtime resources provided by Vercel and the hosted database/storage providers. [2]

For **client-side access**, users need:

- A desktop or laptop computer
- A modern browser such as Chrome, Edge, or Firefox
- Stable internet connectivity
- Recommended minimum **8 GB RAM** for comfortable multitasking
- Recommended display resolution of **1366×768 or higher**

For **development machines**, the following is recommended:

- Minimum **Intel Core i5 / Ryzen 5** or equivalent
- Minimum **8 GB RAM**
- Recommended **16 GB RAM**
- At least **10–20 GB free disk space**
- Node.js runtime, package manager, Git, and code editor installed

For **server-side hosting**, Vercel abstracts hardware management. Therefore, exact server hardware does not need to be specified in the traditional sense. However, the deployed system should assume:

- Managed Node.js execution environment for frontend and backend functions on Vercel
- Managed PostgreSQL (Neon) database for persistent storage
- Managed Cloudinary storage and delivery for files and media

If the system grows significantly in size or traffic, the production environment may later require:

- Upgraded Vercel plan
- Upgraded Neon plan
- Higher Cloudinary usage limits
- Optional background processing service for heavy jobs

7.3 Network Diagram

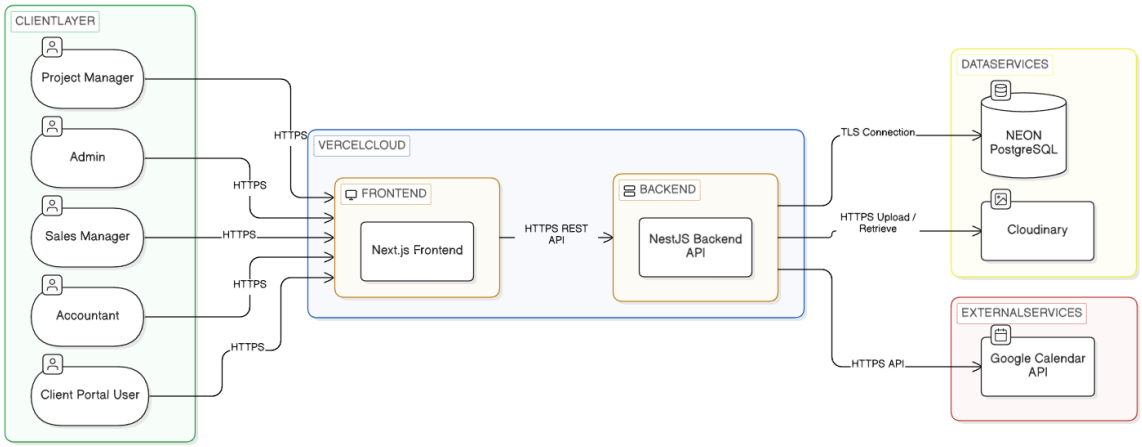


Figure 26: Network Diagram

The network design is based on secure HTTPS communication between users, Vercel-hosted services, PostgreSQL (Neon), Cloudinary, and configured third-party APIs. Users access the application through the Vercel-hosted frontend. The frontend communicates with the backend through REST APIs over HTTPS. The backend then connects to PostgreSQL (Neon) for data persistence, Cloudinary for file storage, and the Google Calendar API for configured scheduling functionality. [2] [5] [6]

This design minimizes infrastructure complexity while keeping the system modular and secure. It also ensures that all core interactions occur through authenticated API calls using JWT-based access control in the backend layer.

7.4 Environment Specification

7.4.1 Development Environment

The development environment is intended for local feature development and debugging by developers. In this environment, the frontend runs locally using the Next.js development server, and the backend runs locally as a **NestJS** application. Developers may use either a local PostgreSQL instance or a separate cloud-based development database. Cloudinary can be connected using a development cloud name and API credentials for testing uploads. Environment variables are stored locally and must not be committed to source control.

Development characteristics

- Local frontend URL: http://localhost:3000
- Local backend URL: http://localhost:5000
- Local or cloud-based PostgreSQL development database
- Development Cloudinary credentials
- Test Google Calendar integration where configured
- Debug logging enabled

7.4.2 Staging Environment

The staging environment is used for integration testing, user acceptance testing, and pre-release validation. It should closely resemble production but use isolated services and credentials. Vercel preview deployments are well-suited for this purpose because Vercel automatically creates preview deployments for non-production branches and pull requests.

Staging characteristics

- Deployed from develop branch or pull requests
- Preview or staging frontend URL on Vercel
- Preview or staging backend deployment on Vercel
- Separate PostgreSQL staging databases.
- Separate Cloudinary folder/preset or separate cloud environment

- Limited external integrations or sandbox credentials
- QA logging and validation enabled

7.4.3 Production Environment

The production environment is the live system used by real users. It must use stable deployment branches, secure secrets management, production-grade database settings, and production Cloudinary credentials. Vercel production deployments are typically linked to the main production branch. Preview deployments can later be promoted or replaced through controlled deployment workflows.

Production characteristics

- Deployed from main branch
- Production frontend hosted on Vercel with custom domain
- Production backend hosted on Vercel with stable API domain
- Production PostgreSQL database/cluster
- Production Cloudinary account or asset namespace
- Configured Google Calendar integration
- Monitoring, alerting, and secure secret management enabled

Chapter 8 References

[1] LangChain, “LangChain Documentation.” [Online]. Available: https://docs.langchain.com/. Accessed: Mar. 18, 2026.

[2] Vercel, “Next.js Documentation.” [Online]. Available: https://nextjs.org/docs. Accessed: Mar. 18, 2026.

[3] OpenJS Foundation, “Node.js Documentation.” [Online]. Available: https://nodejs.org/en/docs. Accessed: Mar. 18, 2026.

[4] NestJS, “NestJS – A progressive Node.js framework.” [Online]. Available: https://nestjs.com/. Accessed: Mar. 18, 2026.

[5] Neon, “Serverless Postgres: Neon database platform.” [Online]. Available: https://neon.tech/. Accessed: Mar. 18, 2026.

[6] Google Developers, “Google Calendar API Documentation.” [Online]. Available: https://developers.google.com/calendar. Accessed: Mar. 18, 2026.

[7] ConstructPro Team, "ConstructPro ERP - Figma Design File," 2026. [Online]. Available: https://www.figma.com/design/07hOsTeEEmnxEOrvasFuJY/DesignProject-Figma?node-id=34-4822&t=JUo5LsesyQvVtDL3-1. Accessed: Apr. 18, 2026.